

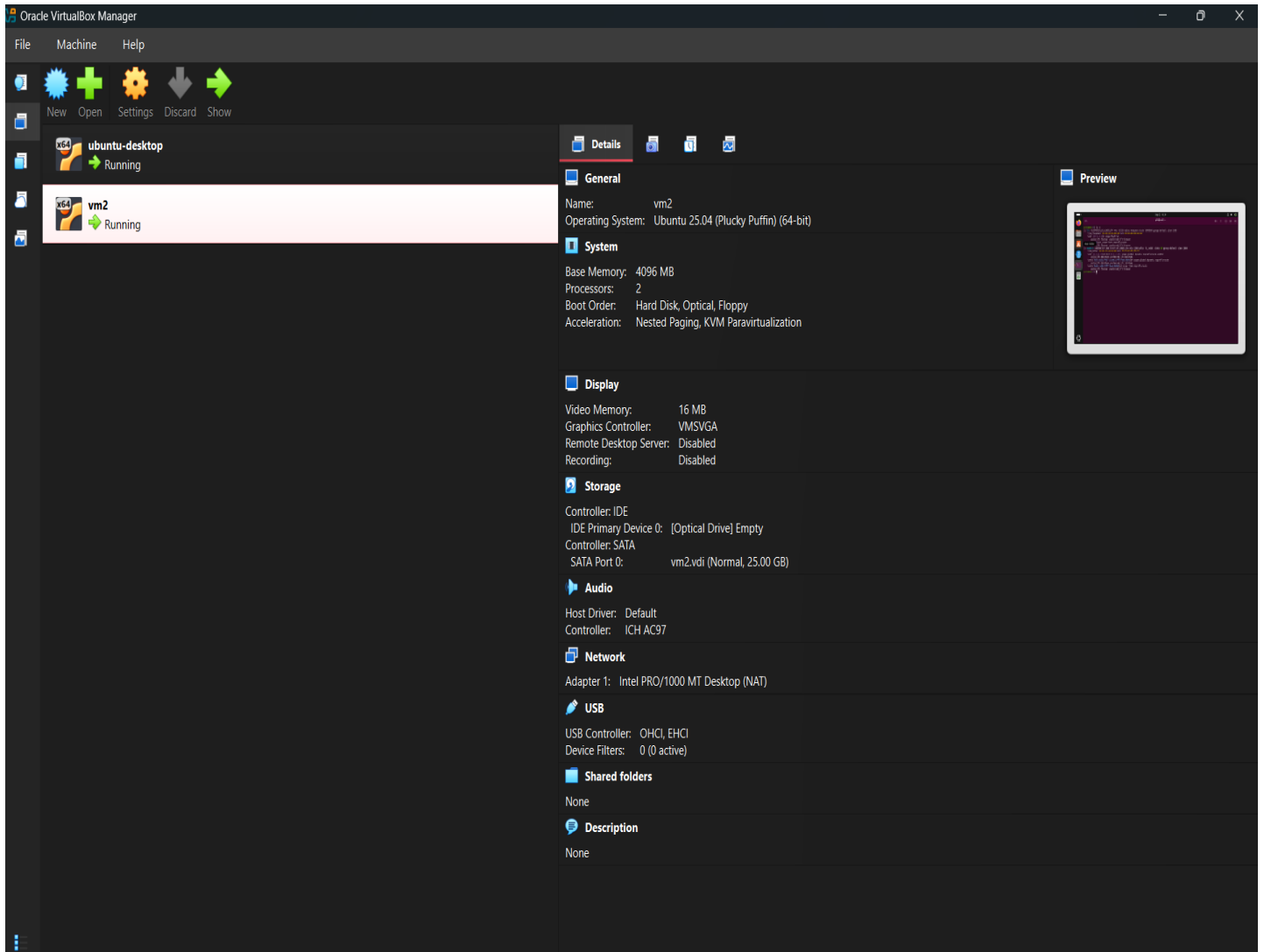
Experiment 2: Client–Server Communication Between Two Virtual Machines

Configure two virtual machine instances in VirtualBox to establish client–server communication and implement a basic TCP-based chat application between the two virtual machines.

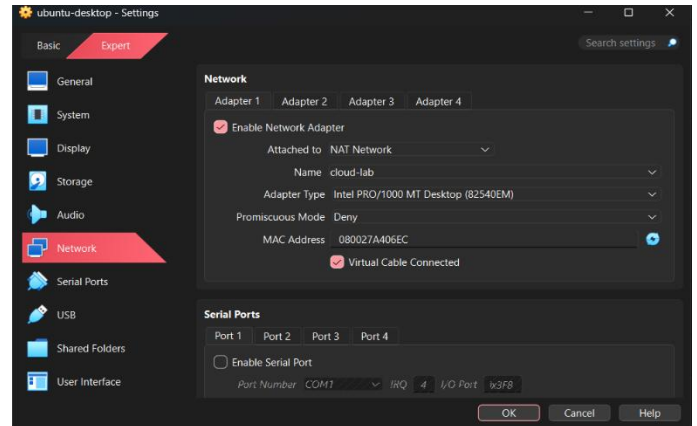
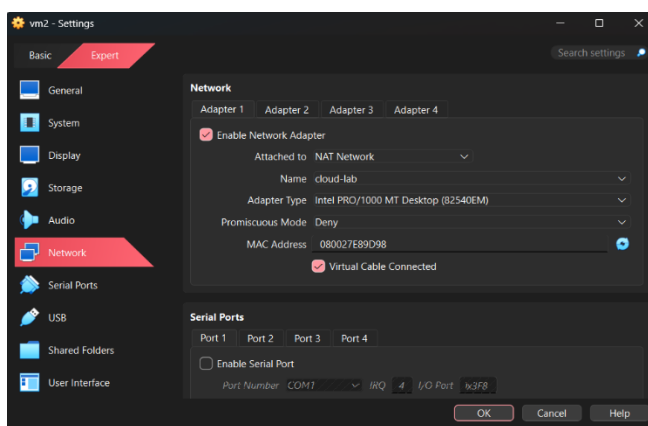
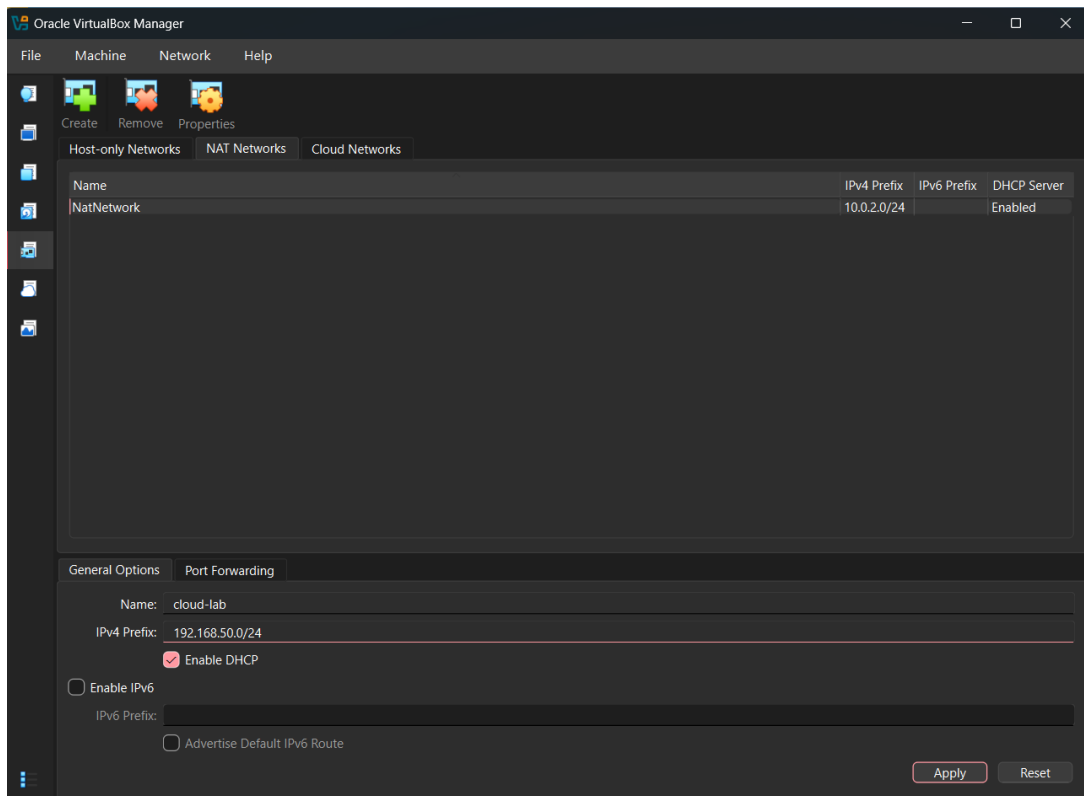
Objective

- Create/prepare two Ubuntu Virtual Machines in VirtualBox.
- Configure both VMs on the same virtual network.
- Assign/check the IP address of both VMs.
- Test connectivity using the ping command.
- Designate one VM as the Server.
- Designate the other VM as the Client.
- Create a TCP server program on the Server VM.
- Create a TCP client program on the Client VM.
- Establish a connection between the two VMs.
- Send and receive messages through the chat application.
- Demonstrate the working chat application.
- Capture screenshots of configuration, connectivity and chat communication.

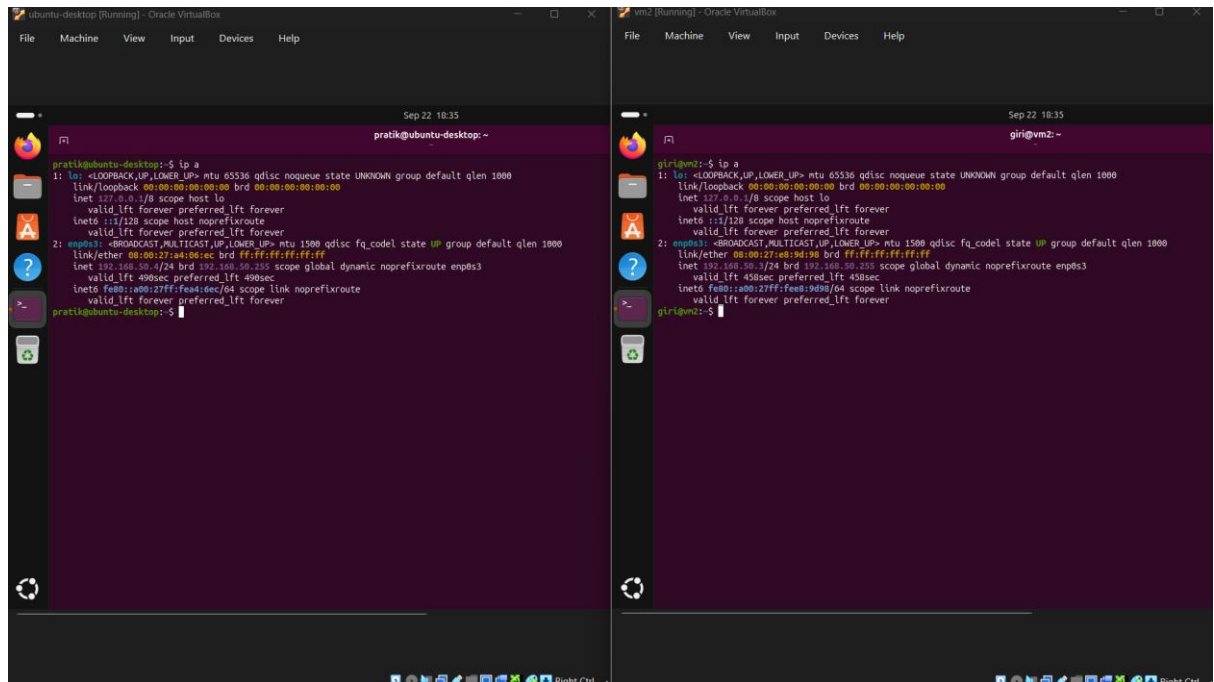
1. Creating and preparing two Ubuntu Virtual Machines in Virtualbox.



2. Configure both VMs on the same virtual network



3. Checking the IP address of both VMs.

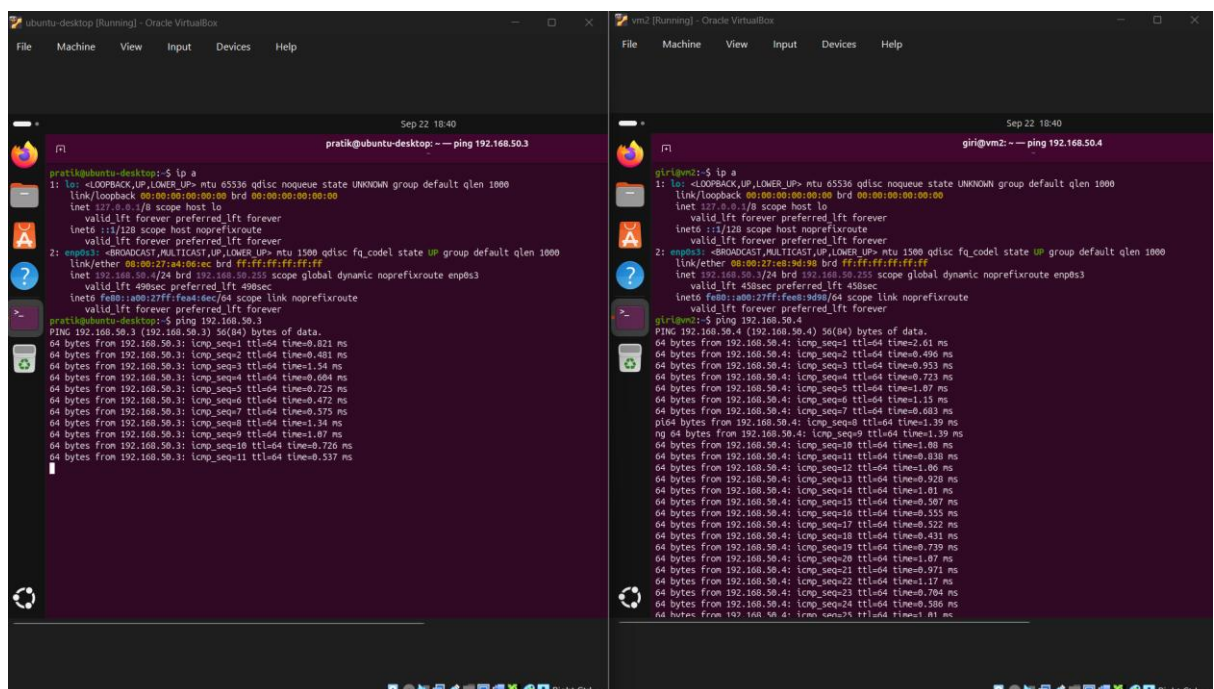


The image shows two terminal windows side-by-side. The left window is titled 'pratik@ubuntu-desktop' and the right window is titled 'giri@vm2'. Both windows show the output of the 'ip a' command, displaying network interface details for 'lo', 'eth0', and 'ens3'. The IP address for 'ens3' in both VMs is 192.168.50.3.

```
pratik@ubuntu-desktop:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:a4:0d:0c brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.3/24 brd 192.168.50.255 scope global dynamic noprefixroute ens3
        valid_lft 490sec preferred_lft 490sec
    inet6 fe80:a00:27ff:fe0d:0c00/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

giri@vm2:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:a4:0d:0c brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.3/24 brd 192.168.50.255 scope global dynamic noprefixroute ens3
        valid_lft 450sec preferred_lft 450sec
    inet6 fe80:a00:27ff:fe0d:0c00/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

4. Testing connectivity using the ping command

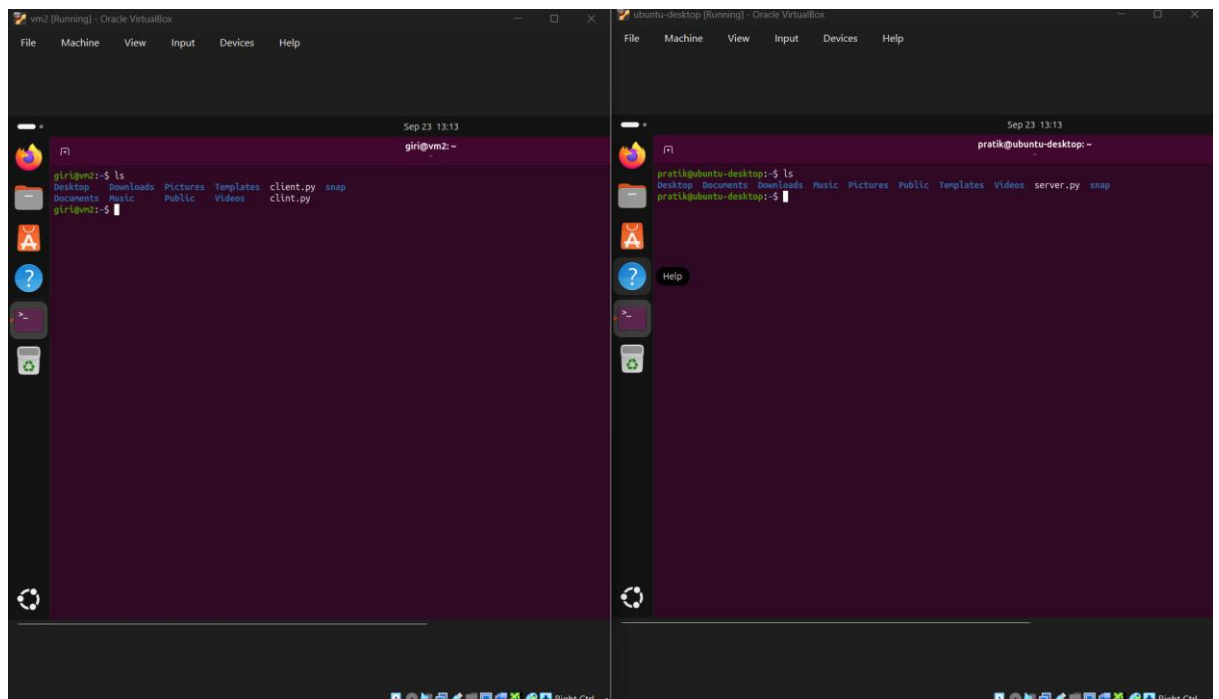


The image shows two terminal windows side-by-side. The left window is titled 'pratik@ubuntu-desktop' and the right window is titled 'giri@vm2'. Both windows show the output of the 'ping' command, testing connectivity to the IP address 192.168.50.3. The ping results show successful connectivity with various statistics including packet size, sequence number, TTL, and time.

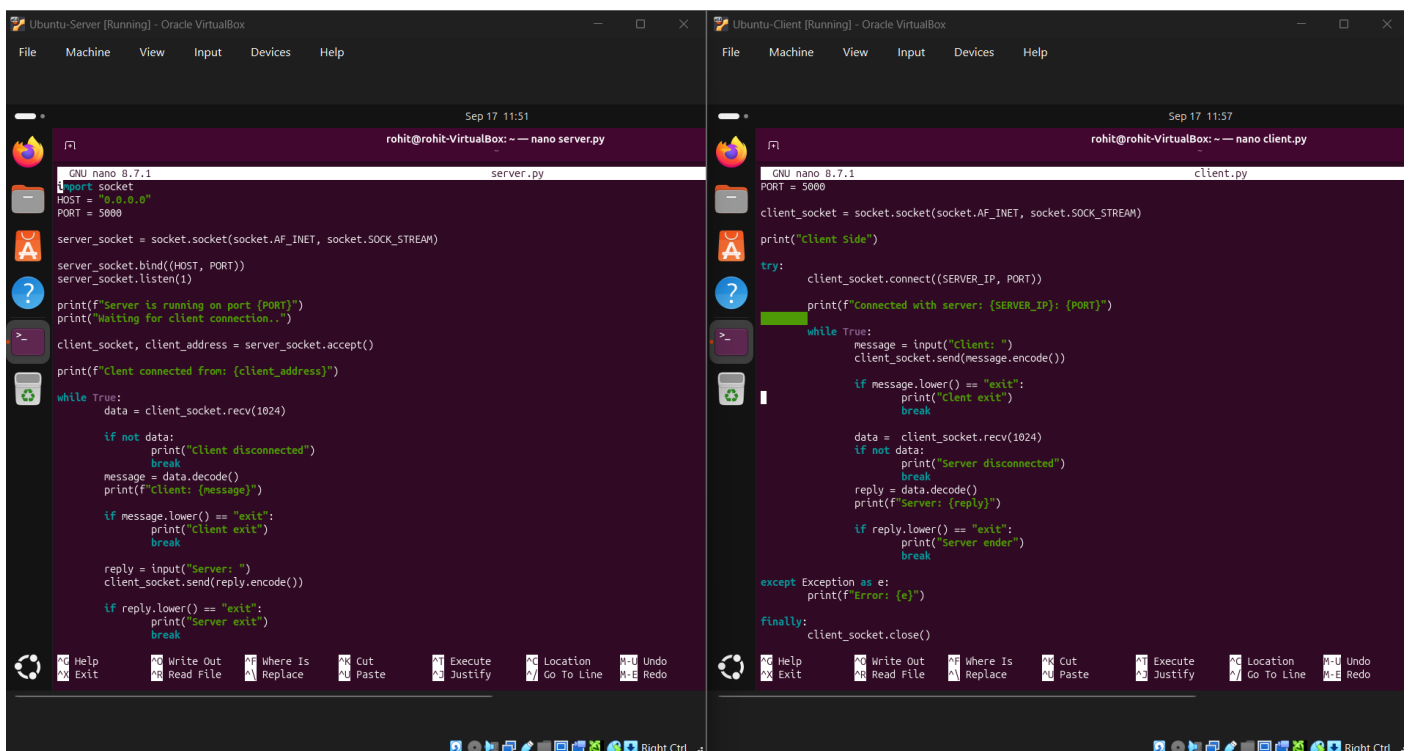
```
pratik@ubuntu-desktop:~$ ping 192.168.50.3
PING 192.168.50.3 (192.168.50.3) 56(84) bytes of data.
 64 bytes from 192.168.50.3: icmp_seq=1 ttl=64 time=0.821 ms
 64 bytes from 192.168.50.3: icmp_seq=2 ttl=64 time=0.481 ms
 64 bytes from 192.168.50.3: icmp_seq=3 ttl=64 time=1.54 ms
 64 bytes from 192.168.50.3: icmp_seq=4 ttl=64 time=0.684 ms
 64 bytes from 192.168.50.3: icmp_seq=5 ttl=64 time=0.725 ms
 64 bytes from 192.168.50.3: icmp_seq=6 ttl=64 time=0.472 ms
 64 bytes from 192.168.50.3: icmp_seq=7 ttl=64 time=0.575 ms
 64 bytes from 192.168.50.3: icmp_seq=8 ttl=64 time=1.34 ms
 64 bytes from 192.168.50.3: icmp_seq=9 ttl=64 time=0.726 ms
 64 bytes from 192.168.50.3: icmp_seq=10 ttl=64 time=0.537 ms

giri@vm2:~$ ping 192.168.50.4
PING 192.168.50.4 (192.168.50.4) 56(84) bytes of data.
 64 bytes from 192.168.50.4: icmp_seq=1 ttl=64 time=2.61 ms
 64 bytes from 192.168.50.4: icmp_seq=2 ttl=64 time=0.496 ms
 64 bytes from 192.168.50.4: icmp_seq=3 ttl=64 time=0.553 ms
 64 bytes from 192.168.50.4: icmp_seq=4 ttl=64 time=0.723 ms
 64 bytes from 192.168.50.4: icmp_seq=5 ttl=64 time=1.97 ms
 64 bytes from 192.168.50.4: icmp_seq=6 ttl=64 time=1.15 ms
 64 bytes from 192.168.50.4: icmp_seq=7 ttl=64 time=0.083 ms
 64 bytes from 192.168.50.4: icmp_seq=8 ttl=64 time=1.39 ms
 64 bytes from 192.168.50.4: icmp_seq=9 ttl=64 time=1.08 ms
 64 bytes from 192.168.50.4: icmp_seq=10 ttl=64 time=0.838 ms
 64 bytes from 192.168.50.4: icmp_seq=11 ttl=64 time=1.06 ms
 64 bytes from 192.168.50.4: icmp_seq=12 ttl=64 time=0.928 ms
 64 bytes from 192.168.50.4: icmp_seq=13 ttl=64 time=1.01 ms
 64 bytes from 192.168.50.4: icmp_seq=14 ttl=64 time=0.507 ms
 64 bytes from 192.168.50.4: icmp_seq=15 ttl=64 time=0.555 ms
 64 bytes from 192.168.50.4: icmp_seq=16 ttl=64 time=0.522 ms
 64 bytes from 192.168.50.4: icmp_seq=17 ttl=64 time=0.431 ms
 64 bytes from 192.168.50.4: icmp_seq=18 ttl=64 time=0.739 ms
 64 bytes from 192.168.50.4: icmp_seq=19 ttl=64 time=1.07 ms
 64 bytes from 192.168.50.4: icmp_seq=20 ttl=64 time=0.971 ms
 64 bytes from 192.168.50.4: icmp_seq=21 ttl=64 time=1.17 ms
 64 bytes from 192.168.50.4: icmp_seq=22 ttl=64 time=0.704 ms
 64 bytes from 192.168.50.4: icmp_seq=23 ttl=64 time=0.586 ms
 64 bytes from 192.168.50.4: icmp_seq=24 ttl=64 time=0.81 ms
```

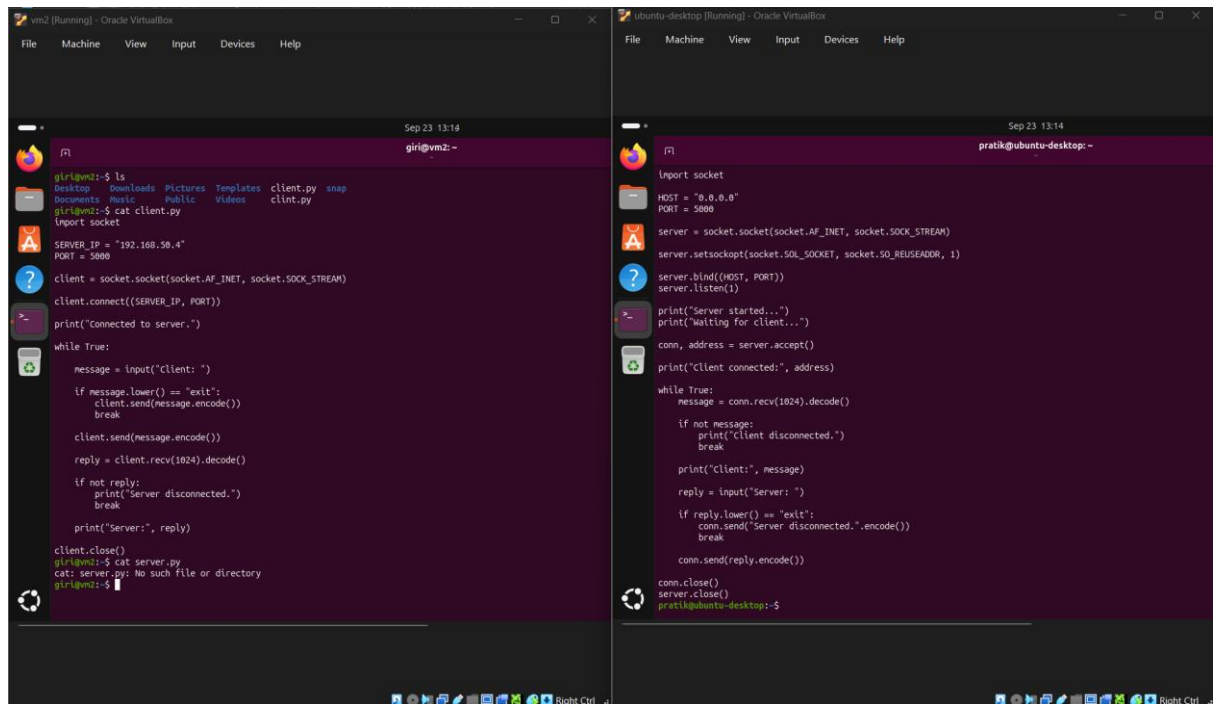
5. Designate one VM as the Server and other as Client.



6. Create a TCP server program on the Server VM and Client VM



7. Establishing a connection between the two VMs.



The screenshot shows two VirtualBox VMs side-by-side. The left VM, named 'vm2', is running a terminal with the following code:

```
giri@vm2:~$ ls
desktop  downloads  Pictures  Templates  client.py  snap
documents  music    public    Videos
giri@vm2:~$ cat client.py
import socket

SERVER_IP = "192.168.50.4"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((SERVER_IP, PORT))
print("Connected to server.")

while True:
    message = input("client: ")
    if message.lower() == "exit":
        client.send(message.encode())
        break

    client.send(message.encode())
    reply = client.recv(1024).decode()

    if not reply:
        print("Server disconnected.")
        break

    print("Server:", reply)

client.close()
giri@vm2:~$ cat server.py
cat: server.py: No such file or directory
giri@vm2:~$
```

The right VM, named 'ubuntu-desktop', is running a terminal with the following code:

```
pratik@ubuntu-desktop:~$
import socket

HOST = "0.0.0.0"
PORT = 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind((HOST, PORT))
server.listen(1)

print("Server started...")
print("Waiting for client...")
conn, address = server.accept()
print("Client connected:", address)

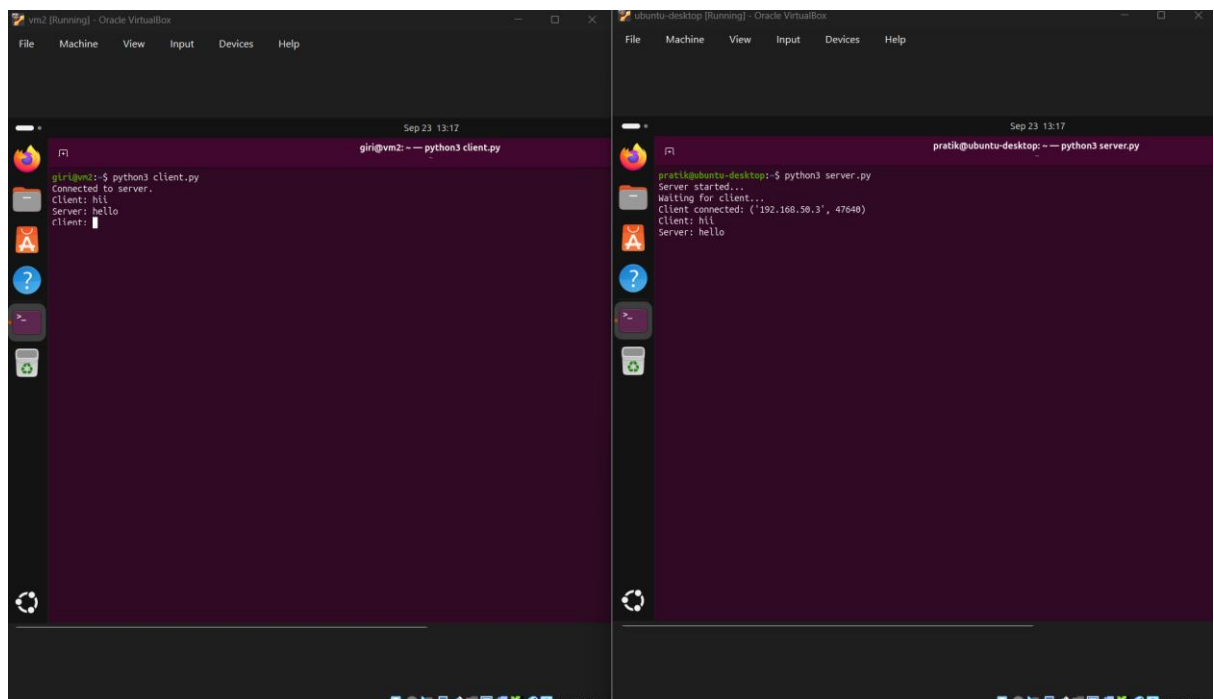
while True:
    message = conn.recv(1024).decode()
    if not message:
        print("Client disconnected.")
        break

    print("Client:", message)
    reply = input("Server: ")
    if reply.lower() == "exit":
        conn.send("Server disconnected.".encode())
        break

    conn.send(reply.encode())

conn.close()
server.close()
pratik@ubuntu-desktop:~$
```

8. Sending and receiving messages through the chat application.



The screenshot shows the same two VirtualBox VMs. The left VM, 'vm2', is running a terminal with the following code:

```
giri@vm2:~$ python3 client.py
Connected to server.
Client: hi!
Server: hello
Client:
```

The right VM, 'ubuntu-desktop', is running a terminal with the following code:

```
pratik@ubuntu-desktop:~$ python3 server.py
Server started...
Waiting for client...
Client connected: ('192.168.50.3', 47648)
Client: hi!
Server: hello
```